Start coding or generate with AI.

Double-click (or enter) to edit

```
## Python is widely used because it’s simple to learn, highly readable, and incredibly versatile—whether you
##  analyzing data, automating tasks, or developing AI. Its huge ecosystem of
##  libraries and strong community support make it
##  beginner-friendly yet powerful enough for complex,
 ## real-world applications, which is why it’s often the
 ## first choice for students,
 ## professionals, and tech companies alike.

## Python - practice code

age = 19


height = 5.9
name = "Kanishk"


is_student = True


x, y, z = 10, 20, 30

a = b = c = 100

# Printing variables
print(age)
print(height)
print(name)
print(is_student)
```

```
19
5.9
Kanishk
True
```

```
type(name)
```

```
str
```

```
x = 10
print(type(x))

name = "Python"
print(len(name))


age = input("Enter your age: ")
print(age)

a = "25"
b = int(a)
c = float(b)
d = str(c)

print(b, c, d)
```

```
<class 'int'>
6
```

```
va1 = "Machine Learning"
len(va1)
```

```
print(va1[-3])
```

```
print(va1[0:10])
```

```
## List is a collection of objects which is ordered and mutable
## vector
## inbuilt function : append
student = [ 'Angelina', 'Abhijeet','Marc', 'Jena ', 'Kanishk' ]
type(student)
```

```
student.append("Mohit")
```

```
print(student)
```

```
type(student)
```

```
student[1] = "Kishore"
student
```

```
student[3] = "Srineedhi"
student
```

```
del student[6]
student
```

```
student.append(10)
```

```
student
```

```
student.count("Kanishk")
```

```
list(range(0,21,2))
```

```
myt = (1,2,3,["Ranm", "Shyam", 21, 22],3,45)
myt[3]
```

```
MyDict = { "Steve" : 90, "Kanishk" : 94, "Rohan" : 92, "Sachin" : 93}
MyDict
```

```
print(MyDict["Steve"])
```

```
type(MyDict)
```

```
print(MyDict)
```

```
## 1. Confusion Matrix Metrics (Python as a Calculator)

TP = 50

FP = 8

TN = 32

FN = 10

Total Predictions  = TP + FP + TN  + FP = 100  # Formula - (a)

Accuracy = (TP + TN)/Total = 82/100 = 82% # Formula - (b)

Precision = TP/(TP+FP) = 50/(50+8) = 50/58 = 86.2% # Formula - (c)

Recall(Sensitivity) = TP/(TP+FN) = 50/(50+10) = 83.33% # Formula - (d)

F1 Score = 2*Precision*Recall/(Precision+Recall) = 2*0.862*0.833/(0.862+0.833) = 84.7% # Formula - (e)

##  2. Calculating R² (Coefficient of Determination)
```

```
•   Sum of Squares due to Regression (SSR) = 420
•   Sum of Squares due to Error (SSE) = 180

SST = SSR(420) + SSE(180) = 600
R² = SSR/SST = 420/600 = 70%

## The model explains 70% of the data, but not more than seventy percent.

## 3. Mean Squared Error (MSE) Using Python as Calculator

# Store squared errors
Errors = [2, 4, 1, 9, 4]

# Calculate Mean Squared Error
MSE = sum(Errors) / len(errors)

# Print result
print("Mean Squared Error (MSE):", MSE)

## 4. Cook’s Distance (Influential Observation)

Squared residual = 9

Mean Squared Error = 3

Mean Squared Regression = 3

Number of predictors = 3

Sample size = 25

## Calculation :

 Denominator = MSE*(1-h)**2 = 3*(1-0.2)**2 =  3(0.64) = 1.92

 Numerator = (9/4)*0.2 = 2.25*0.2 = 0.45

 D = 0.45/1.92 = 0.234
```

```
 Threshold Comparison : 4/n = 4/25 = 0.16

D(Cook's distance) = 0.234 > 0.16

Hence, the observation is influential.

## 5. Simple Linear Regression Prediction

y = 2.5x + 10

y = 2.5(12) + 10 = 40

# 6. Regression model: y = 2.5x + 10
x = 12
y = 2.5 * x + 10

print("Predicted crop yield:", y)

Predicted crop yield: 40

Slope (2.5):

This tells us how much crop yield increases for each unit increase in rainfall.

Here, for every 1 unit of rainfall, crop yield increases by 2.5 units.

Intercept (10):

This is the baseline crop yield when rainfall is zero.

It represents the yield contributed by other factors (soil fertility, farming practices, etc.) even without ra
```